Lekcja 7: Python – funkcje i programowanie funkcyjne

`#lekcja` `#python` `#funkcje` `#programowanie-funkcyjne` `#dekoratory`

W tej lekcji poznamy paradygmat programowania funkcyjnego w Pythonie. Dowiemy się, jak pisać bardziej zwięzły, elegancki i przewidywalny kod, traktując funkcje jak obywateli pierwszej kategorii. Opanujemy potężne narzędzia, takie jak list comprehensions, funkcje lambda, map, filter, a także zaawansowane koncepcje, takie jak domknięcia i dekoratory.

1. List Comprehensions (Wyrażenia listowe)

Definition

List comprehension to zwięzły, jednolinijkowy sposób tworzenia nowej listy na podstawie istniejącej sekwencji (lub innej iterowalnej kolekcji). Jest to często bardziej czytelna i szybsza alternatywa dla tradycyjnej pętli for.

Pełna składnia: `[wyrażenie for element in sekwencja if warunek]`

Tradycyjna pętla vs. List Comprehension:

```
# Tradycyjny sposób
kwadraty_liczb_parzystych = []
for i in range(1, 11):
    if i % 2 == 0:
        kwadraty_liczb_parzystych.append(i ** 2)
print(kwadraty_liczb_parzystych)
# Wynik: [4, 16, 36, 64, 100]

# Sposób z list comprehension
kwadraty_liczb_parzystych_v2 = [i ** 2 for i in range(1, 11) if i % 2 == 0]
print(kwadraty_liczb_parzystych_v2)
# Wynik: [4, 16, 36, 64, 100]
```

Comprehensions dla innych typów

Tę samą składnię można zastosować do tworzenia słowników i zbiorów:

```
# Set comprehension (tworzy zbiór unikalnych wartości)
kwadraty_zbior = {x ** 2 for x in [1, 2, 2, 3]}
# Wynik: {1, 4, 9}
```

```
# Dict comprehension (tworzy słownik)
liczby_slownik = {x: x ** 2 for x in range(1, 5)}
# Wynik: {1: 1, 2: 4, 3: 9, 4: 16}
```

2. Funkcje lambda

> **Note**
>
> Funkcja lambda (nazywana też funkcją anonimową) to mała, jednolinijkowa funkcja bez nazwy. Jest idealna do prostych operacji, gdzie definiowanie pełnej funkcji za pomocą def byłoby nadmiarowe.

Składnia: `lambda argumenty: wyrażenie`

Funkcja lambda może mieć dowolną liczbę argumentów, ale tylko **jedno wyrażenie**, którego wynik jest automatycznie zwracany.

```
# Tradycyjna funkcja
def dodaj(a, b):
    return a + b

# Odpowiednik w postaci funkcji lambda
dodaj_lambda = lambda a, b: a + b

print(dodaj(2, 3))       # 5
print(dodaj_lambda(2, 3)) # 5
```

Główne zastosowanie: Lambdy najczęściej używane są jako argumenty dla funkcji wyższego rzędu, takich jak `sorted()`, `map()` czy `filter()`.

```
# Sortowanie listy krotek po drugim elemencie
produkty = [("mleko", 2.50), ("chleb", 3.80), ("jajka", 6.20)]
posortowane_po_cenie = sorted(produkty, key=lambda produkt: produkt[1])
print(posortowane_po_cenie)
```

3. Funkcje wyższego rzędu

> **Info**
>
> Funkcja wyższego rzędu (Higher-Order Function) to funkcja, która spełnia co najmniej jeden z poniższych warunków:

1. Przyjmuje jedną lub więcej funkcji jako argumenty.
2. Zwraca funkcję jako swój wynik.

W Pythonie funkcje są "obywatelami pierwszej kategorii", co oznacza, że można je traktować jak każde inne dane: przypisywać do zmiennych, przekazywać jako argumenty i zwracać z innych funkcji.

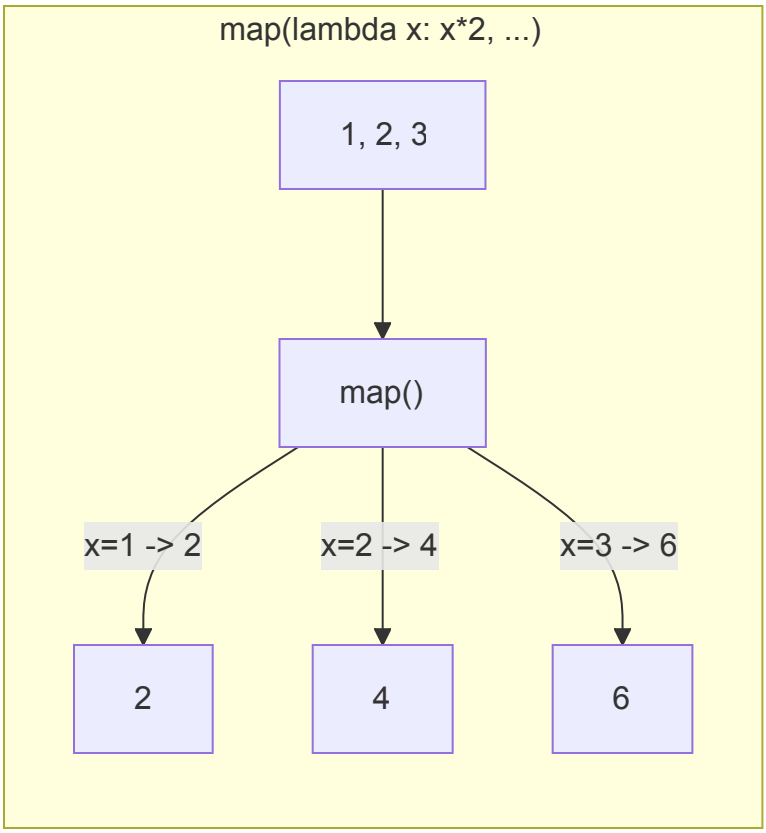
```
# Przykład funkcji, która przyjmuje funkcję jako argument
def wykonaj_operacje(funkcja, a, b):
    """Wykonuje podaną funkcję na dwóch argumentach."""
    return funkcja(a, b)

wynik = wykonaj_operacje(lambda x, y: x * y, 5, 4)
print(f"Wynik operacji: {wynik}") # Wynik operacji: 20
```

4. `map`, `filter`, `reduce`

Są to klasyczne funkcje wyższego rzędu, które pozwalają na eleganckie przetwarzanie kolekcji.

`map()` – transformacja każdego elementu

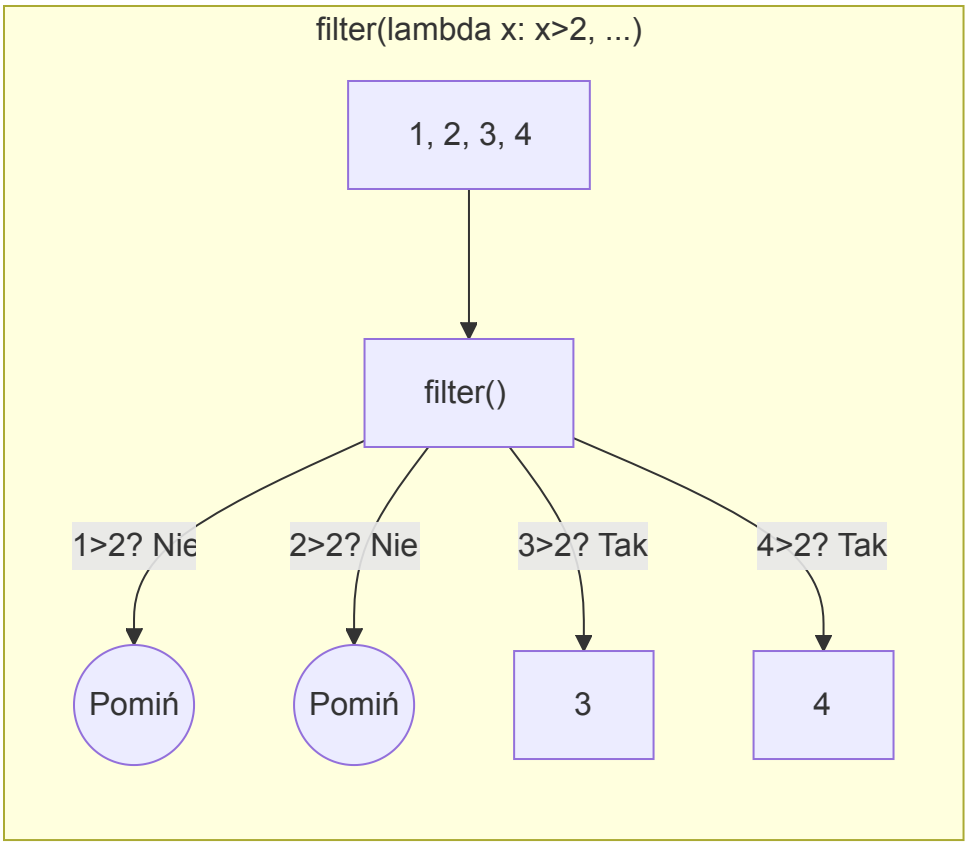
`map(funkcja, sekwencja)` stosuje podaną `funkcję` do każdego elementu `sekwencji` i zwraca iterator z wynikami.




```
liczby = [1, 2, 3, 4]
podwojone_liczby = list(map(lambda x: x * 2, liczby))
print(podwojone_liczby) # [2, 4, 6, 8]
```

`filter()` – filtrowanie elementów

`filter(funkcja, sekwencja)` zwraca iterator z tymi elementami `sekwencji`, dla których `funkcja` zwróciła `True`.




```
liczby = [1, 2, 3, 4, 5, 6]
parzyste = list(filter(lambda x: x % 2 == 0, liczby))
print(parzyste) # [2, 4, 6]
```

`reduce()` – redukcja do jednej wartości

`reduce(funkcja, sekwencja)` stosuje `funkcję` kumulatywnie do elementów sekwencji, redukując ją do pojedynczej wartości. Funkcja musi przyjmować dwa argumenty (akumulator i bieżący element).

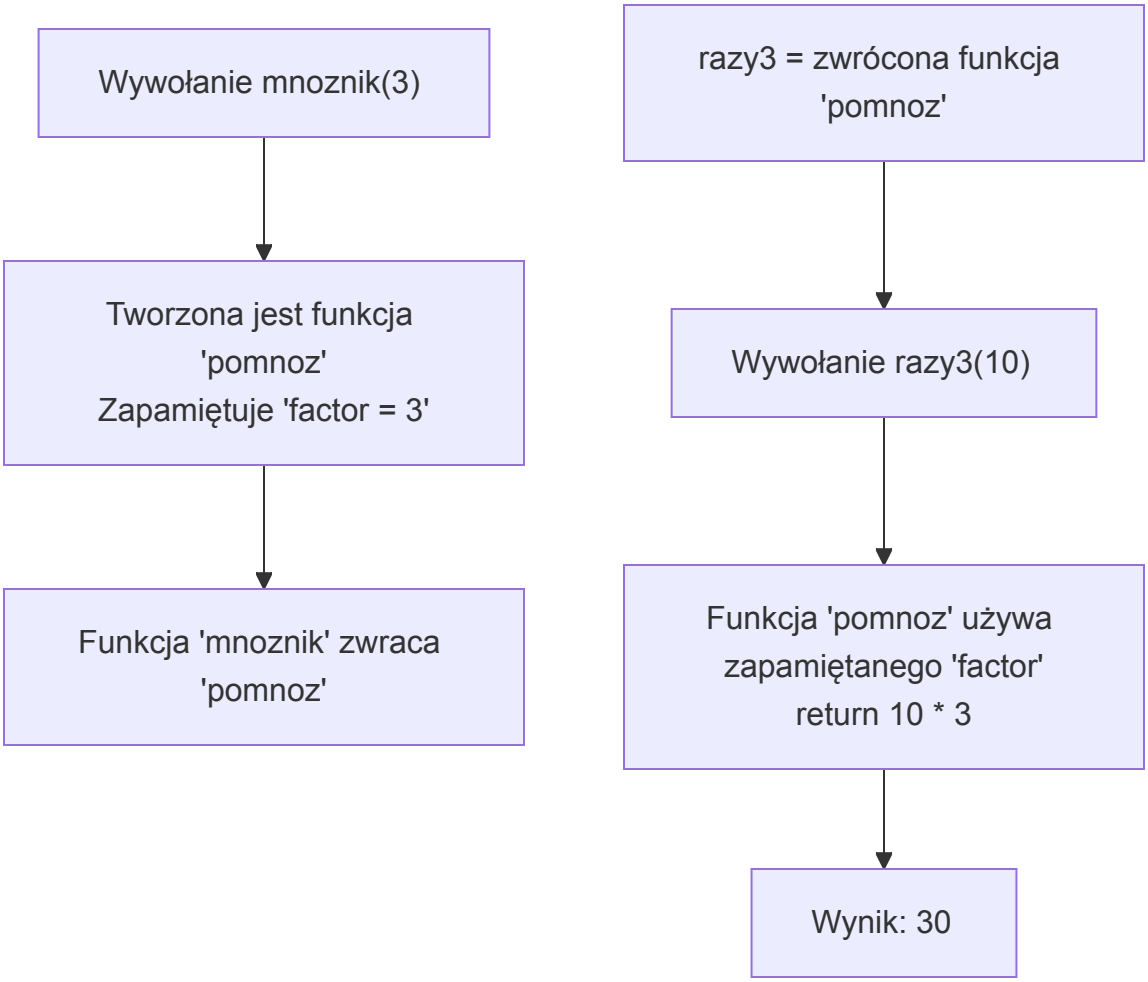
> **Tip**
>
> reduce() znajduje się w module functools, więc trzeba go zaimportować.

```
from functools import reduce

liczby = [1, 2, 3, 4]
suma = reduce(lambda akumulator, element: akumulator + element, liczby)
print(suma) # 10 ((((1+2)+3)+4))
```

5. Domknięcia (Closures)

Definition

Domknięcie (closure) to mechanizm, w którym funkcja wewnętrzna "pamięta" i ma dostęp do zmiennych z otaczającej ją funkcji (zakresu nielokalnego), nawet po tym, jak funkcja zewnętrzna zakończyła swoje działanie.




```python
def fabryka_mnoznikow(czynnik):
    """To jest funkcja fabryczna, która tworzy i zwraca inne funkcje."""
    def mnoz(liczba):
        # Ta funkcja 'pamięta' wartość 'czynnik' z otaczającego zakresu
        return liczba * czynnik
    return mnoz

# Tworzymy funkcję, która będzie mnożyć przez 3
razy3 = fabryka_mnoznikow(3)
# Tworzymy funkcję, która będzie mnożyć przez 10
razy10 = fabryka_mnoznikow(10)
```

```
print(razy3(5))   # 15
print(razy10(5))  # 50
```

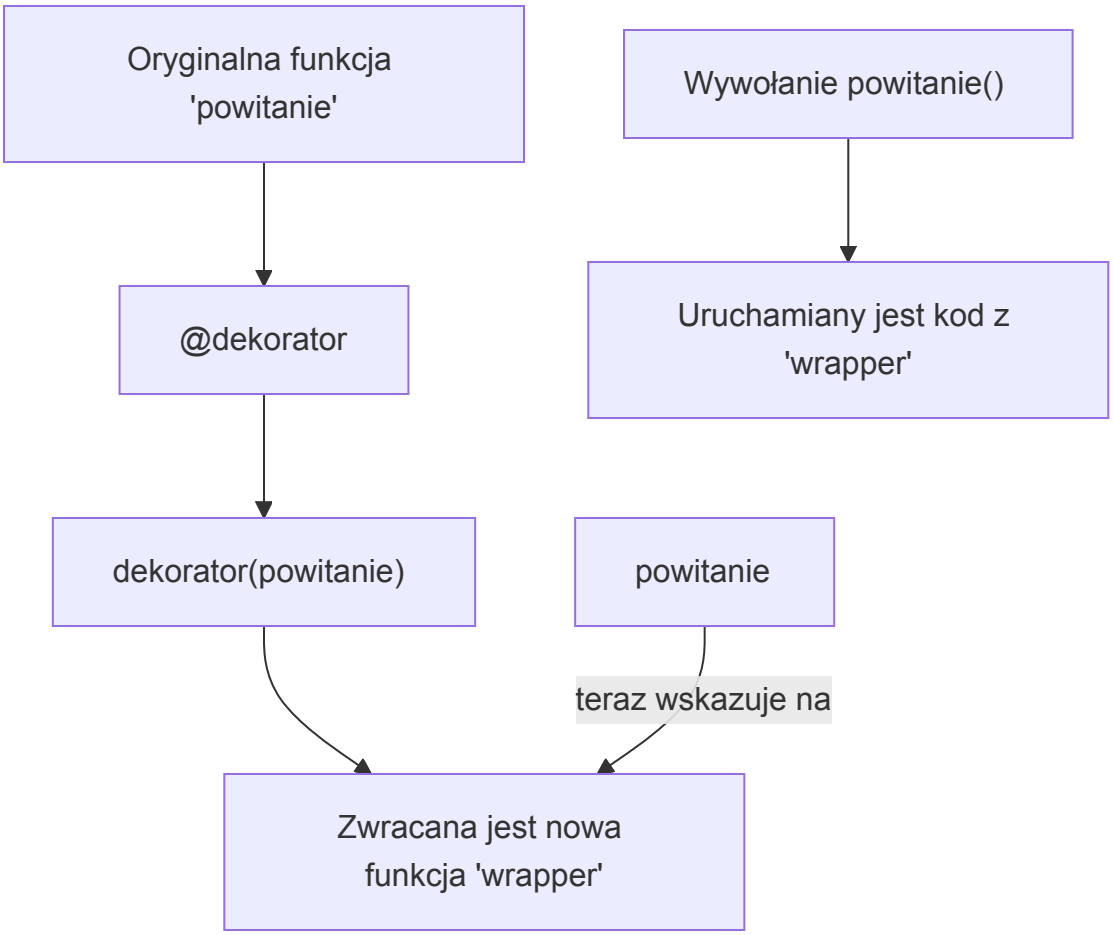
6. Dekoratory

> **Note**
>
> Dekorator to specjalny rodzaj funkcji, która przyjmuje inną funkcję jako argument, "opakowuje" ją w dodatkową logikę i zwraca zmodyfikowaną funkcję, nie zmieniając jej oryginalnego kodu.

Dekoratory używają składni ze znakiem `@` (tzw. "cukier składniowy").

Jak to działa?

Zapis @moj_dekorator nad funkcją funkcja_dekorowana jest równoważny z funkcja_dekorowana = moj_dekorator(funkcja_dekorowana).



Przykład praktyczny – mierzenie czasu wykonania:

```python
import time

def mierz_czas(funkcja):
    """Dekorator mierzący czas wykonania funkcji."""
    def wrapper(*args, **kwargs):
        start = time.time()
        wynik = funkcja(*args, **kwargs) # Wywołanie oryginalnej funkcji
        koniec = time.time()
        print(f"Funkcja '{funkcja.__name__}' wykonała się w {koniec - start:.4f} sek.")
        return wynik
    return wrapper

@mierz_czas
def duze_obliczenia():
    """Funkcja, która symuluje długie obliczenia."""
    time.sleep(1)
    print("Obliczenia zakończone!")

duze_obliczenia()
```

🧪 7. Zadania do samodzielnej pracy

1. **Filtrowanie słów:** Mając listę słów `slowa = ["jabłko", "banan", "kiwi", "gruszka", "truskawka"]`, użyj **list comprehension**, aby stworzyć nową listę zawierającą tylko te słowa, które mają więcej niż 5 liter.
2. **Sortowanie słownika:** Masz słownik `oceny = {"Jan": 4, "Anna": 5, "Piotr": 3, "Kasia": 4}`. Użyj funkcji `sorted()` i **funkcji lambda**, aby posortować elementy słownika (klucz, wartość) według ocen (od najwyższej do najniższej).
3. **Konwersja na wielkie litery:** Użyj funkcji `map()`, aby przekształcić listę imion `imiona = ["anna", "piotr", "kasia"]` w listę imion pisanych wielką literą.
4. **Znajdowanie liczb pierwszych:** Użyj funkcji `filter()`, aby z listy liczb od 1 do 30 wybrać tylko liczby pierwsze. (Wskazówka: napisz pomocniczą funkcję `czy_pierwsza(n)`, która sprawdza, czy liczba jest pierwsza).
5. **Iloczyn elementów:** Użyj funkcji `reduce()`, aby obliczyć iloczyn (wynik mnożenia) wszystkich liczb w liście `[1, 2, 3, 4, 5]`.
6. **Licznik wywołań:** Stwórz **domknięcie** (closure). Napisz funkcję `stworz_licznik()`, która zwraca funkcję. Każde wywołanie zwróconej funkcji powinno zwiększać wewnętrzny licznik i zwracać jego aktualną wartość.
7. **Dekorator logujący:** Napisz dekorator `@loguj`, który przed wywołaniem udekorowanej funkcji wypisze komunikat `Uruchamiam funkcję [nazwa_funkcji]...`, a po jej zakończeniu `Zakończono funkcję [nazwa_funkcji].`.

8. **Łączenie map i filter:** Mając listę liczb `[-5, 2, 8, -1, 0, 10]`, użyj `filter` do wybrania tylko liczb dodatnich, a następnie `map` do obliczenia ich kwadratów. Zrób to w jednej linijce.
9. **Dekorator z argumentem:** Stwórz dekorator `@powtorz(n)`, który przyjmuje argument `n` i powoduje, że udekorowana funkcja zostanie wykonana `n` razy.
10. **Mini-projekt: Przetwarzanie danych:** Masz listę słowników reprezentujących użytkowników:

    ```
    uzytkownicy = [
        {"imie": "Jan", "wiek": 30, "aktywny": True},
        {"imie": "Anna", "wiek": 17, "aktywny": False},
        {"imie": "Piotr", "wiek": 25, "aktywny": True}
    ]
    ```

    Napisz jednolinijkowy kod (używając kombinacji `filter`, `map` lub list comprehension), który zwróci listę imion aktywnych użytkowników, którzy mają 18 lat lub więcej, pisanych wielkimi literami.